

Dev [Boost]

GitHub Copilot

GitHub Copilot Essentials Training

Welcome, About DevBoost and
Introduction to GitHub Copilot,
Features & Best Practices.



Table of contents

• Introductions	3
• Introduction to DevBoost	5
• Introduction to GitHub Copilot	11
• GitHub Copilot UI & Features	17
• Demo	22
• Prompt Engineering	23
• Some Best Practices	32
• Future Training Topics	38
• Q&A	

Introductions

Lily Tribel

Senior Architect
Slalom, London



Welcome to

Dev [Boost]

GitHub Copilot

Accelerating the Software Development Life Cycle





Vision

Siemens DI aims to **revolutionise software development with Generative AI** by deploying 20,000 **GitHub Copilot licenses to boost productivity**. We are committed to a seamless, secure adoption process, investing in our developer community and training teams.

As the engineering landscape evolves, we are at the forefront, integrating **prompt engineering with traditional skills to shape the next generation of software engineering excellence**. This initiative marks our first step towards AI-accelerated engineering, aiming for **broader productivity gains** across the software development lifecycle. With strategic support from Slalom and collaboration with GitHub, we are dedicated to creating a roadmap for **wider GenAI adoption and greater value realisation**.

Dev [Boost]

GitHub Copilot



User-Centric Learning

Customized Learning Experiences

- Live trainings, self learnings, practical exercises
- From beginner to expert

Fully integrated

- Utilizing Siemens branding and platforms
- Accessible to everyone



Consistent Messaging

One central landing page

- News, updates and info snippets
- Single source of truth on everything GitHub Copilot

Bridging all BUs

- Central communication to all BUs
- Learning from each other



Community Engagement

For Siemens From Siemens

- Activating existing champions
- Bringing forward Siemens success stories

Continuous Engagement

- Share Best Practices
- “Prompt of the Week”
- Regular Community Sessions

No additional cost – training comes with license

Accepting Terms of Use

For you to begin using GitHub Copilot, it is critical to accept the Siemens Terms of Use relating to its usage.

T - Tool: Controlled by developer and developer decides on if and in what form generated code is integrated.

E - Evaluate Proposed Code: Treat the output as a draft and ensure it's reviewed by a human developer.

A - Adapt or Edit (whenever meaningful or required): Significant changes made by the developer will render the code protected as independent work)

M - Manage Use: Developer determines how, where, and in what form the generated code is used.

GitHub Copilot for Business Offered by github.siemens.cloud

Hallo, Thomas. Wenn Sie dieses Formular absenden, sieht der Eigentümer Ihren Namen und Ihre E-Mail-Adresse.

* Erforderlich

Participation request

By clicking on the send button you agree to the information below

1. General information:

This is a service offered by github.siemens.cloud - your internal ACP-3 managed GitHub Enterprise provider!

We are using GitHub Copilot for Business, therefore GitHub Copilot does not store your code and does not learn from your code!

GitHub Copilot Wiki: <https://pages.github.siemens.cloud/ghes-gps/github-enterprise-wiki/documentation/other/copilot/>

Siemens Viva

Engage: <https://engage.cloud.microsoft/main/groups/eyJfdHlwZSI6Ikdvd3VwIiwiaWQ0IixODY1MDg5OTI1MTI0>

Unit approval status:

- Approved: DL, ADV (ADV AI policy applies), SFS, Service & Governance, RC, SI (except SI BSW)
- Waiting for approval: SMO

Costs:

The costs of GitHub Copilot for Business are 13€ per user per month and is charged to the Cost Center of the Department

How can I use GitHub Copilot for Business after I finished this self-service?

After you pressed the send button, you'll get an E-Mail with an exact description.

Terms of Use:

- The approval and distribution of the Copilot licenses is only done by the GitHub Enterprise team via this self-service, don't buy separate subscriptions and/or interact with the GitHub Copilot sales team by yourself!
 - Do not use the subscriptions of GitHub Copilot offered for private users for work purposes!
- Your use of the service is subject to the compliance with the Acceptable Use Policies of GitHub Copilot as published here: <https://docs.github.com/en/site-policy/acceptable-use-policies>. For example, you may not use GitHub Copilot to generate suggestions whose use you know (or reasonably should know) would be unlawful or would infringe on the rights of others.
- Mark generated code: You must mark the suggestions generated by GitHub Copilot as defined here: <https://pages.github.siemens.cloud/ghes-gps/github-enterprise-wiki/documentation/other/copilot/#documentation-of-github-copilot-usage>.
- Security: GitHub Copilot is approved for all ACP levels, but the ACP level of where you host your source code is relevant. Follow the mandatory SFeRA Secure Software Development Standard and Product and Solution Security requirements.
 - Do not blindly trust the generated code, validate and quality check if it's suitable for the intended purpose and following secure coding practices. We recommend to also check the code quality with tools like SonarQube and to have a high test coverage for the Copilot generated code
 - Train the usage of GitHub Copilot and check the Tips & Tricks (see Wiki).
- Export Control:
 - If the export control classification of your product is different than AL=N & ECCN=N or EAR99 then the upload of controlled technology is not allowed on third party servers because of potential license restrictions. Please contact your EX responsible (<https://intranet.legal-compliance.siemens.com/compliance/ico/#icoContacts>) to clarify the usage of GitHub Copilot.
 - Currently, the usage of GitHub Copilot is not allowed in embargo countries (Belarus, Cuba, Iran, North Korea, Russia, Syria)
- These terms are subject to change and will be adapted as necessary, e.g. due to additional legal requirements.

Advanta only:

- I confirm that I successfully passed the mandatory AI training: https://my.learningworld.siemens.com/web/en/app/toc/lex_auth_013995406482554880583/overview

✖

☒ I accept the Terms of Use

Absenden

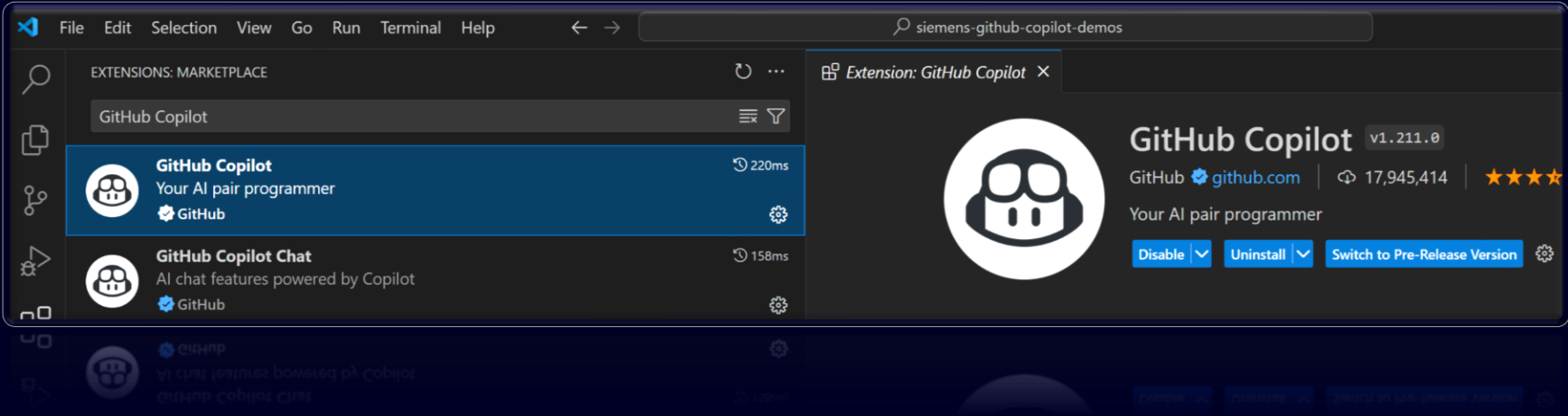
Microsoft 365

Dieser Inhalt wird vom Besitzer des Formulars erstellt. Die von Ihnen übermittelten Daten werden an den Formularbesitzer gesendet. Microsoft ist nicht für die Datenschutz- oder Sicherheitspraktiken seiner Kunden, einschließlich derer dieses Formularbesitzers, verantwortlich. Geben Sie niemals Ihr Kennwort heraus.

Microsoft Forms | KI-gestützte Umfragen, Quizze und Umfragen [Eigene Formulare erstellen](#)

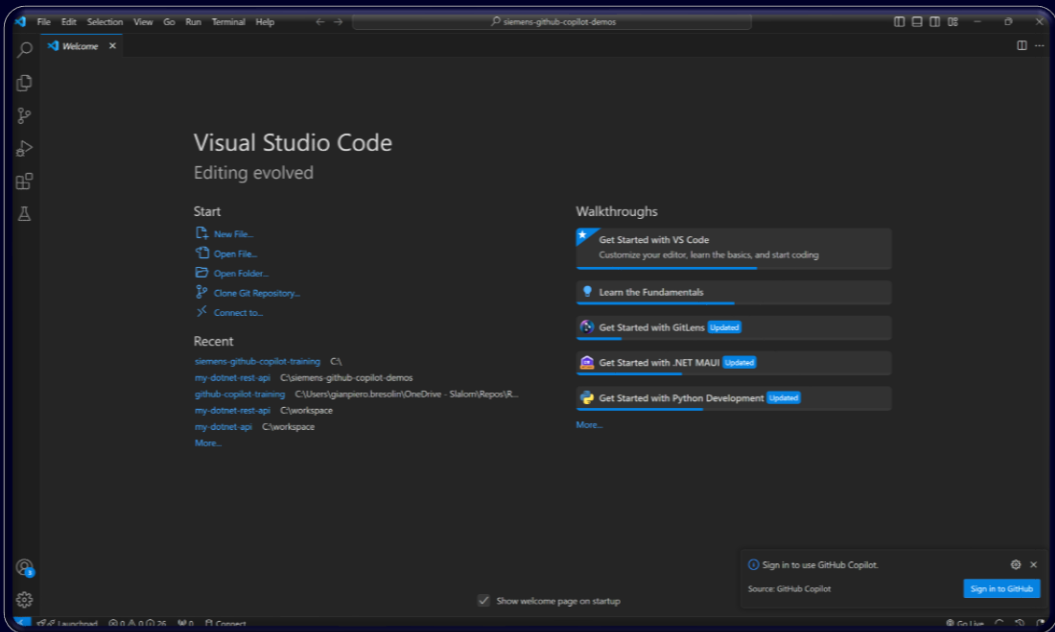
Installation

1 Install GitHub Copilot

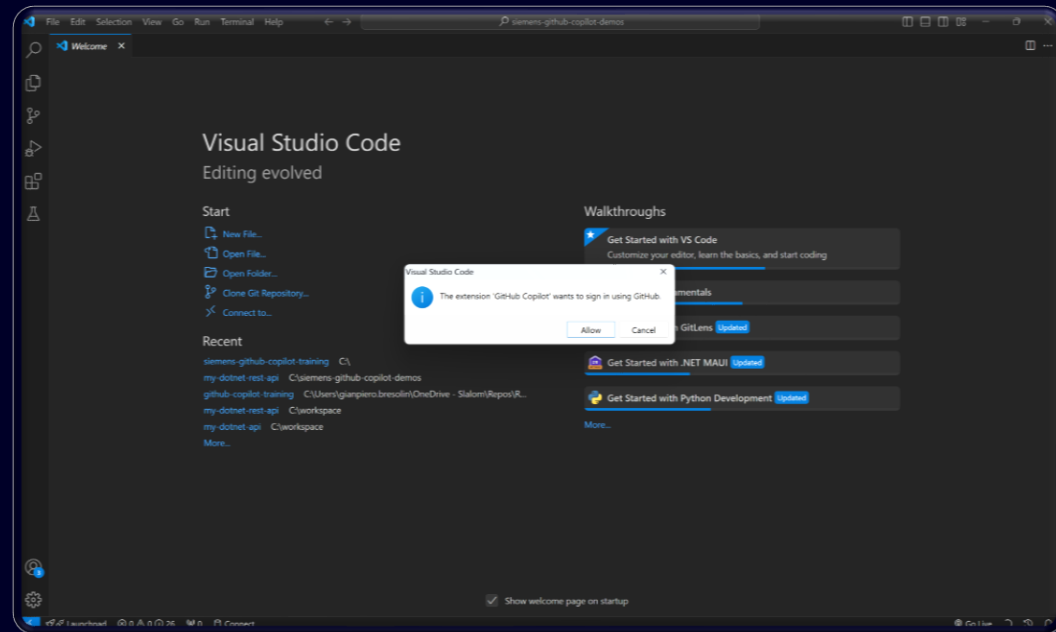


Installation

2 Sign into GitHub Copilot

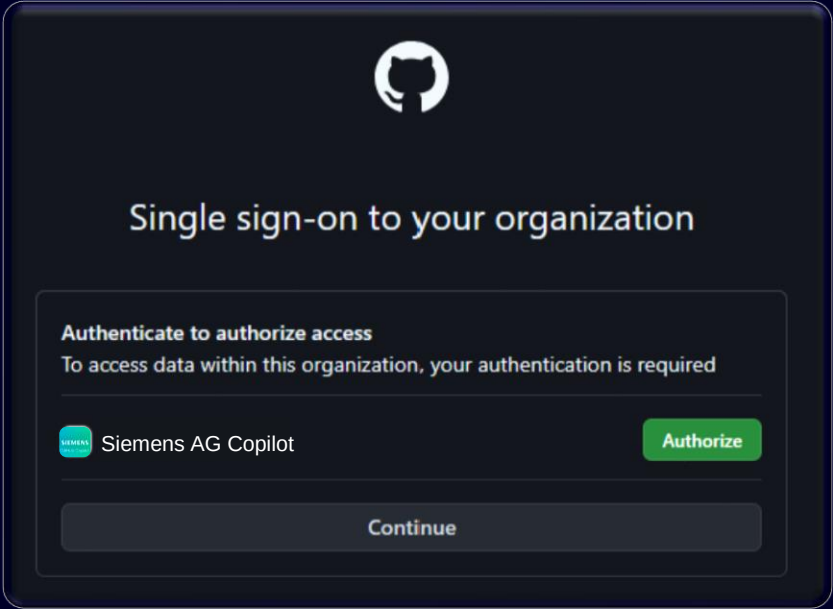


3 GitHub wants to sign in

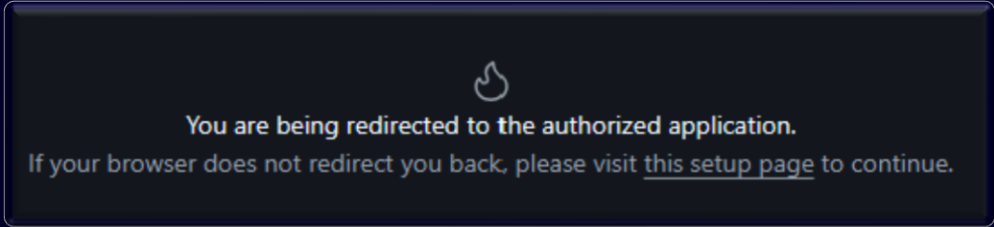


Installation

4 Single Sign On



5 Authorized



Dev [Boost]

GitHub Copilot

Introduction to GitHub Copilot

What it is, what it's not and how it works underneath the hood.



You already know how to program, and you have been doing so successfully for years without GitHub Copilot, so you might be asking yourself: **Why are you here?**

Why are you here?



Increased Productivity:

Developers saw an average increase in productivity of 10%.



Valued internally:

83% of Siemens GitHub Copilot users would recommend it to a colleague.



Benefits For All Developers:

Senior developers got a performance boost of 13.2% & junior developers got a performance boost of 8.5%.



Overall Satisfaction:

59.5% of developers report being satisfied or very satisfied with GitHub Copilot.



Enhanced Learning:

GitHub Copilot is like having a helpful Developer sitting next to you at all times!



What is it?

Integrated development environment extensions that generate code using trained LLMs trained on large volumes of public code. Currently uses GPT-4o (“o” for “omni”).

What GitHub Copilot does

- Generate blocks of code (from one line to whole classes).
- "Understands" the area of the code you're working in.
- Handles boilerplate very quickly.
- Improving rapidly with new features being released each month.
- Copilot is **probabilistic**, not deterministic. This means that you might not always get the exact same result for the same prompts.



What it's not

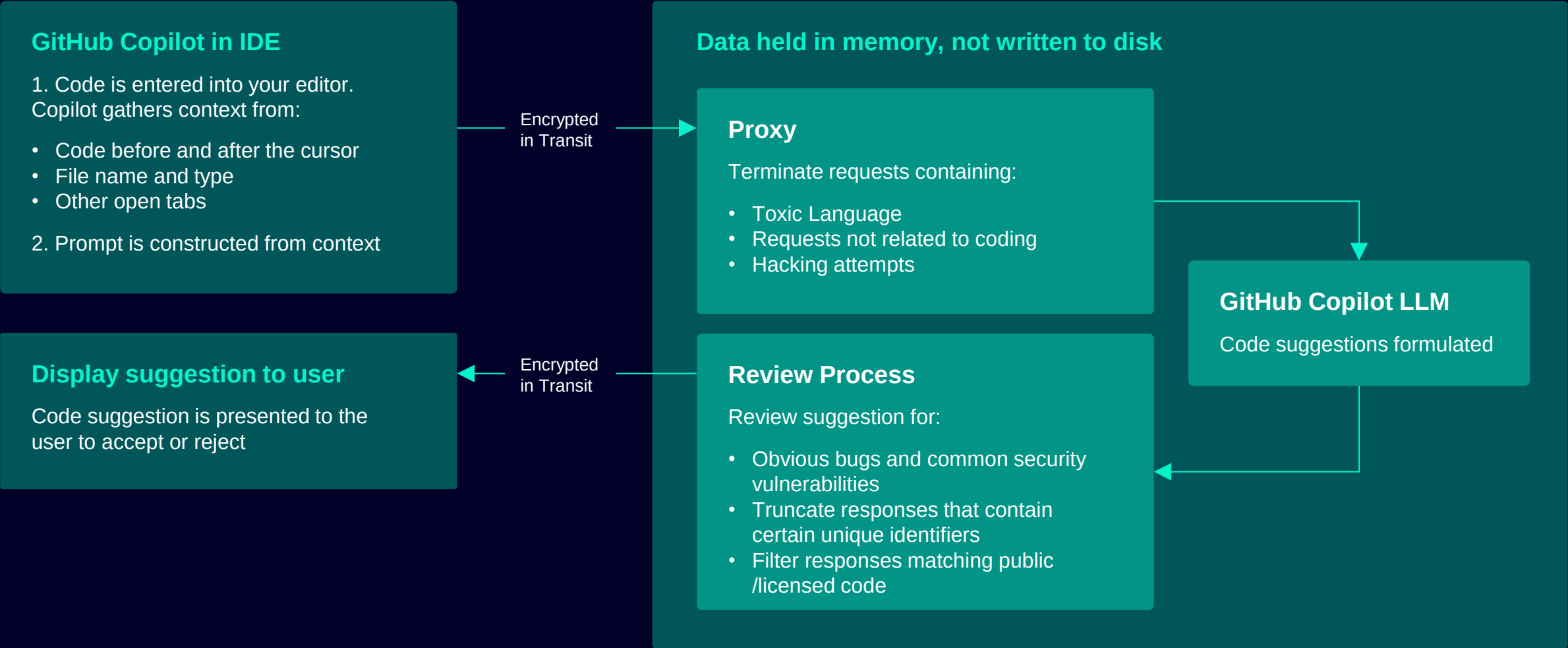
GitHub Copilot is an amazing development tool, but it is just that: a tool. It will not make a bad developer into a good developer, but it can help make a good developer into a great one.

What GitHub Copilot doesn't do:

- Doesn't know about what you are trying to achieve and your overall goals without your help.
- Does not "understand" the full context of an application.
- While code suggested follows best practices, it doesn't know if it has made a mistake in what it has suggested.

In Summary: **It's not the Pilot – YOU ARE!**

GitHub Copilot – How a suggestion is generated



Dev [Boost]

GitHub Copilot

UI & Features

GitHub Copilot, GitHub Copilot Chat,
Inline Chat & Slash Commands



GitHub Copilot:

Offers code suggestions, contextualised to your project



Convert comments to code



Autocompletions & autofill for repetitive code



Show alternatives

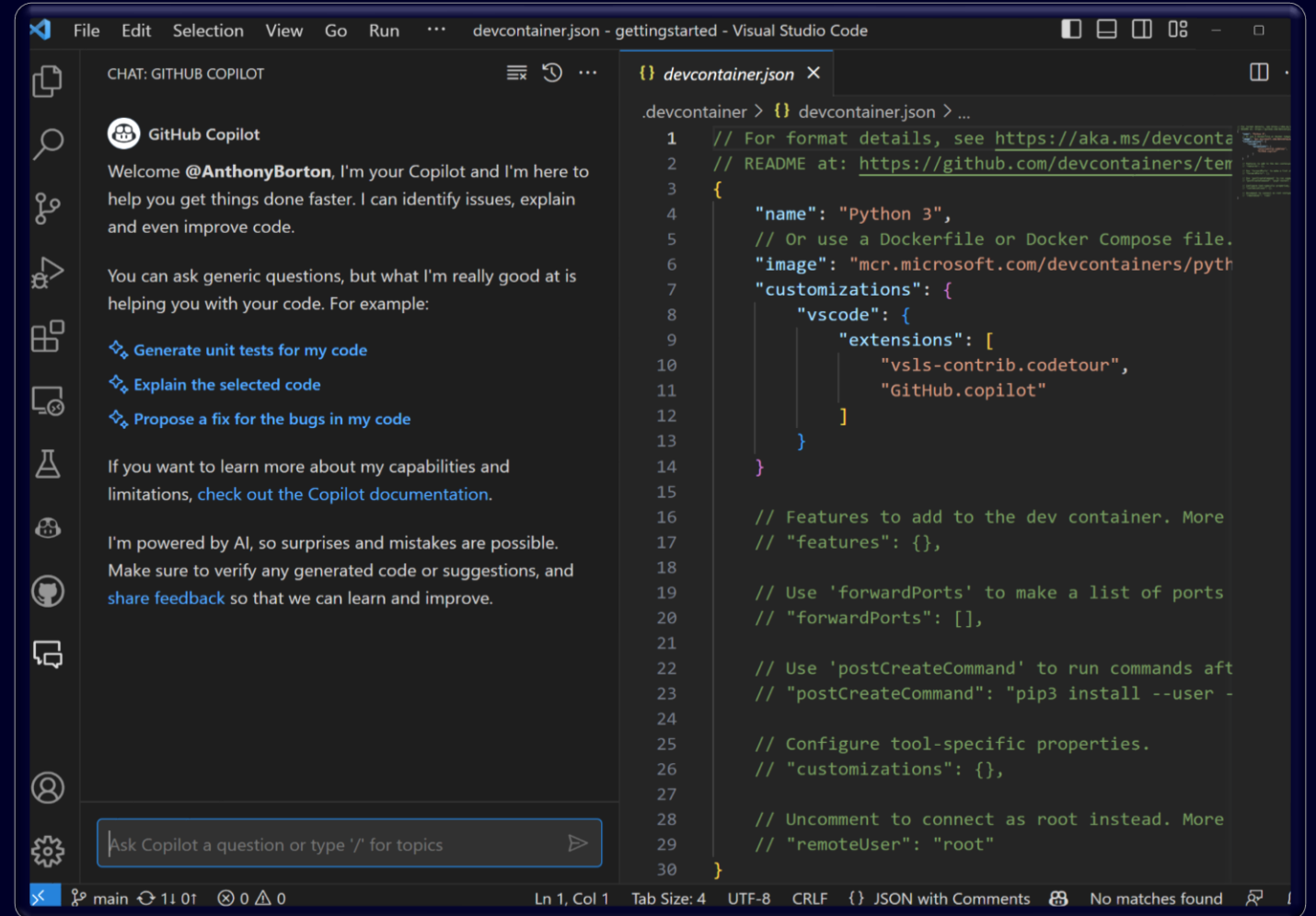
```
1 #!/usr/bin/env ts-node
2
3 import { fetch } from "fetch-h2";
4
5 // Determine whether the sentiment of t
6 // Use a web service
7 async function isPositive(text: string)
8   const response = await fetch(`http://
9     method: "POST",
10    body: `text=${text}`,
11    headers: {
12      "Content-Type": "application/x-www
13    },
14  });
15  const json = await response.json();
16  return json.label === "pos";
17
```

Copilot

GitHub Copilot Chat:

Context-aware ChatGPT-styled conversations with Copilot

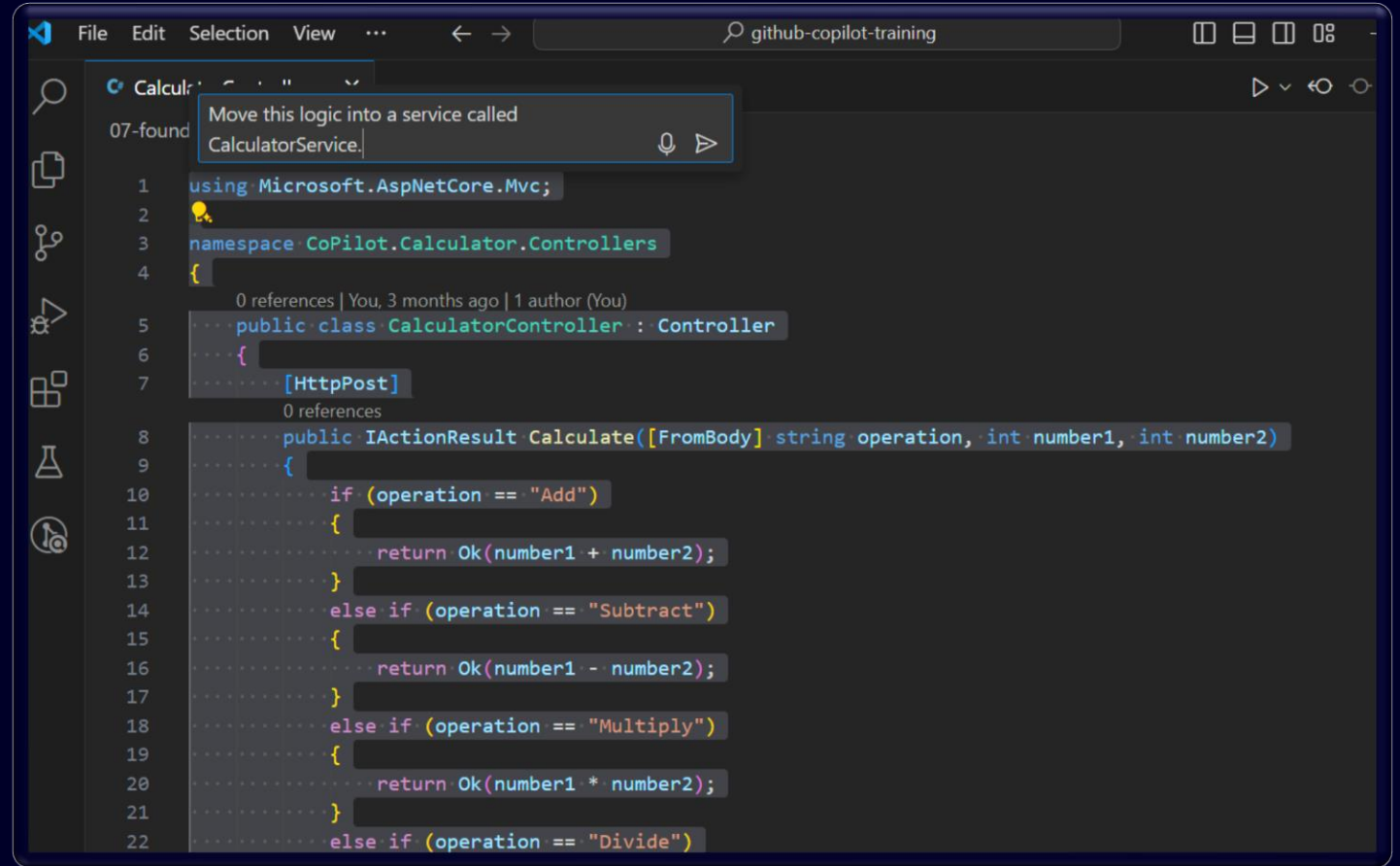
- ✓ Answer coding questions
- ✓ Explain code
- ✓ Translate languages
- ✓ Explain errors in terminal



GitHub Copilot Inline Chat:

Brings Copilot Chat functionality directly into the editor or terminal

- ✓ Generate code from prompts
- ✓ Refactor directly in your code
- ✓ Generate CLI commands
- ✓ Multi-turn conversations
- ✓ Add documentation
- ✓ Write unit tests
- ✓ Propose code fixes



GitHub Copilot Slash Commands:

@workspace agent

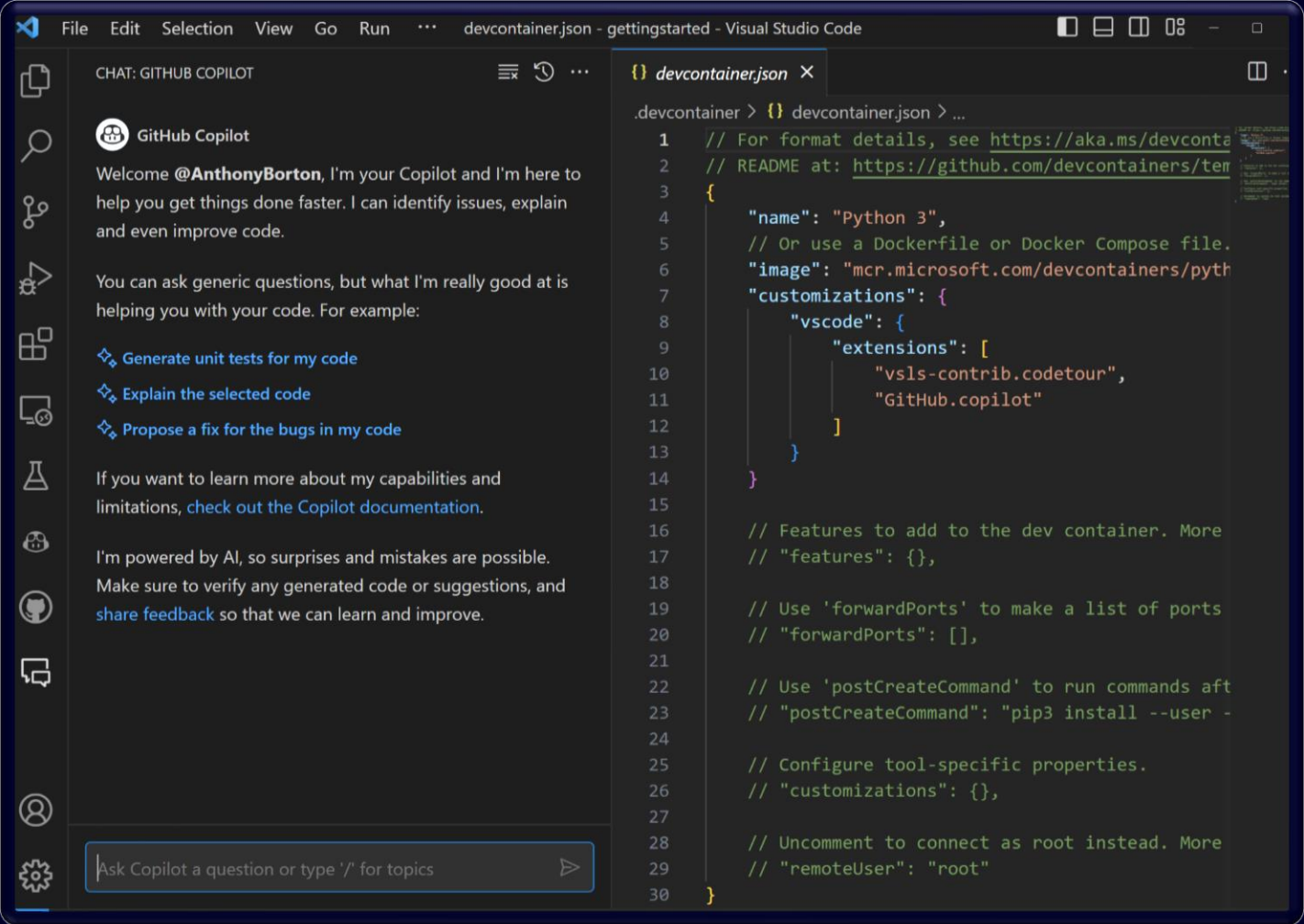
- /explain** Explains how the selected code works
- /fix** Proposes a fix for the problems in selected code
- /new** Create a new workspace or file based on a natural language description
- /newNotebook** Create a new Jupyter Notebook based on your description
- /tests** Generate unit tests for the selected code
- /clear** Clear the current session
- /help** General help about GitHub Copilot

@vscode agent

- /runCommand** Search for and execute a VS Code command
- /search** Generate search queries to find code

@terminal agent

- /explain** Explain terminal errors and commands





Prompt Engineering

Tips, tricks and prompt engineering techniques

What is prompt engineering?

the process of giving clear natural language instructions to computer programs, such as the large language models (LLMs) that drive GitHub Copilot functionalities.

Principles for crafting effective prompts



Know your model

- Different language models have different capabilities, strengths and weaknesses
- Some models are not trained on code, not trained to perform well with complicated reasoning, or not trained on recent data.
- GitHub Copilot is built for purpose as it knows all about coding best practices!



Be detailed and specific

- Provide detailed information about your desired output using direct language
- Focus the prompt on a specific problem, topic, question or output type.
- Multiple asks in a single prompt can make it difficult for a model to answer effectively



Provide context

- Contextual information helps the model to solve the task and generate more appropriate responses
- Your GitHub Copilot chat conversation history, examples of desired output, or the model's role in the prompt aid in ensuring the best response.



Experiment

- Try different approaches to prompting. Crafting effective prompts is more of an art than a science
- Alter prompt wording, change the prompt to align with the model's capabilities, and even ask the model for help to optimize the prompt

Prompt engineering strategies

These strategies are like design patterns in software engineering. They are proven approaches when working with LLMs, that can be used to improve the suggestions offered by the model.

Prompt Engineering Strategies

1

Chain of Thought

Explain your goal explicitly step by step or ask the model to solve problems step by step by breaking the problem down into smaller pieces and solving them individually.

Extremely valuable for reasoning and logic tasks.

Prompt: **Write a Python function to calculate factorial.**



Prompt: **Write a Python function** named 'calculate factorial' that takes an integer 'n' as input. The function should use a for loop to calculate the factorial of 'n'. If 'n' is 0 or 1, the function should return 1.

Prompt Engineering Strategies

2

One Shot/Few Shot

Providing the model with one or more similar examples of the output that you want.

The model uses these examples as “context” or priming to generate its output.

Prompt: **Generate a RegEx pattern to validate phone numbers.**



Prompt: **Generate a RegEx pattern to validate** the following phone numbers: (123) 456-7890, 123-456-7890, 123.456.7890, 123 456 7890, +33 123 456 7890, 1234567890

Prompt Engineering Strategies

3

Generated Knowledge

GitHub Copilot has been trained on a huge amount of knowledge, so by asking Copilot to provide additional information you are enriching the context which improves suggestions.

Ask the model to provide additional useful information so it can generate a better response.

Prompt: **Create a Python script that reads a CSV file and prints each row.**



Prompt: **I need to work with large CSV datasets, can you suggest a library to use and create a python script that reads a CSV file and then prints each row.**

Prompt Engineering Strategies

4

Prompt Chaining

Is a technique where you break the problem down into individual parts or steps, and have the model solve each part individually, and then link the solution together.

Prompt: **Implement feature toggles for a web application.**



Prompt: **Design a database schema to store feature toggle configurations including the feature name, enabled state, and applicable user segments.**

Prompt: **Next, develop a C# middleware in the application that queries this database before each feature is accessed to determine if it should be available to the current user**

Prompt: **Lastly, show me how to use this middleware in my C# web application.**

General Considerations

1. Don't try and create entire application in one prompt.
2. Use the strategies mentioned previously to improve context (one shot/few shot prompting, prompt modifiers, etc.).
3. If you are unsure where to start use the prompt templates for guidance or ask GitHub Copilot for prompt advice.
4. Clearly state what you want to achieve, include relevant keywords.
5. Identify inputs using back ticks (`` ``).
6. Consider the output in the prompt
7. Prompts should be short and concise.
8. Ask Copilot if it has any questions.
9. Don't change topics in the middle of a conversation.
10. Test & experiment.
11. Share prompts that you have found to be effective!



Some Best Practices

Some guidance on getting the best of GitHub Copilot

Context, Context, Context

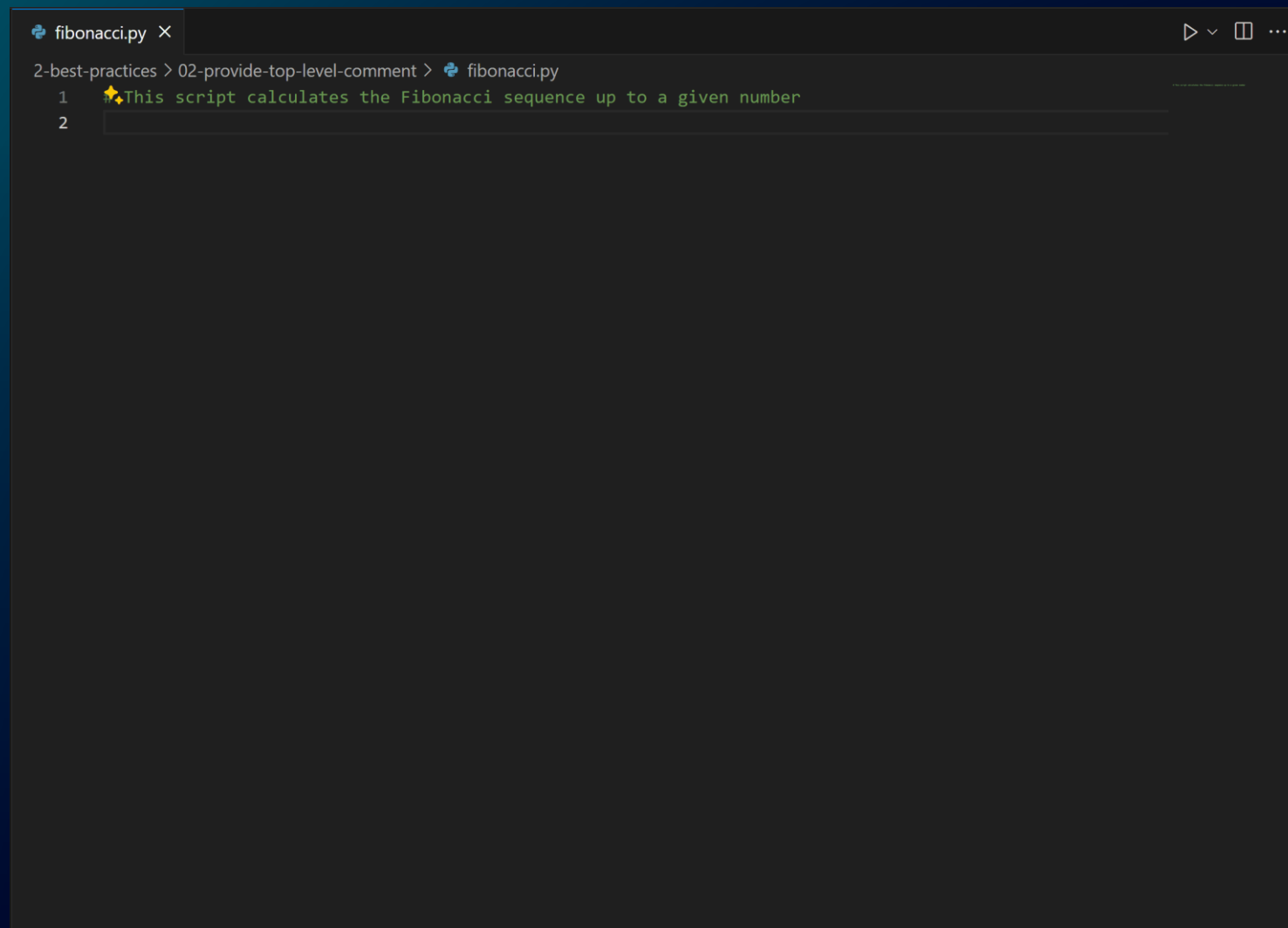
- GitHub Copilot's context is built from the current code you're working on, open files and the prompt.
- It considers the code in your current file, including comments, function names, variable names, and other semantic information.
- This context is critical for Copilot to understand your coding intentions.
- The more context Copilot has, the better it can predict and suggest relevant code.
- Without sufficient context, Copilot's ability to provide accurate and helpful suggestions is significantly reduced.

Best Practices

1

Top Level Comments

By providing clear and concise comments at the beginning of your file, you can help GitHub Copilot understand what you are trying to achieve which will lead to more helpful suggestions.



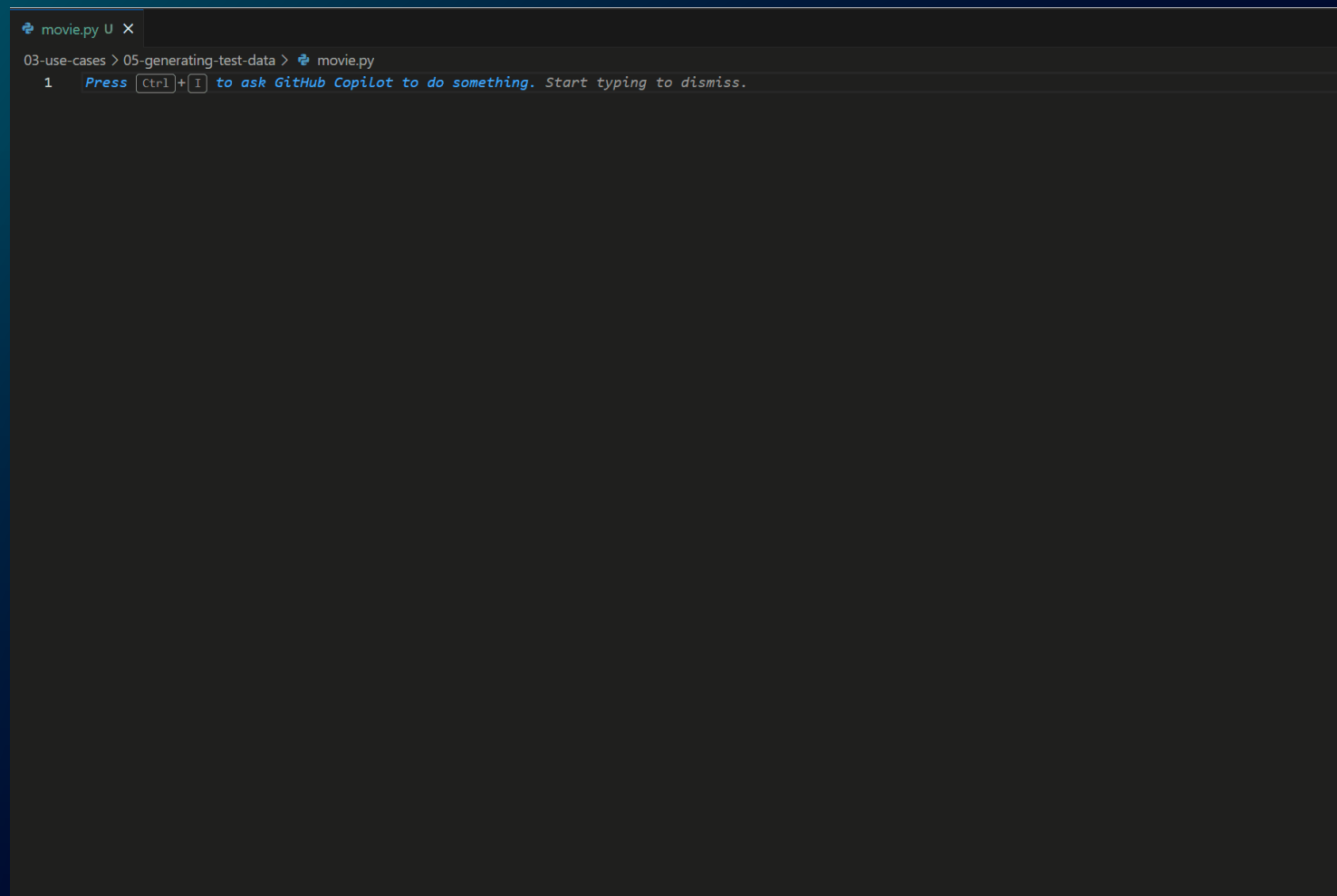
The screenshot shows a code editor window with a tab labeled 'fibonacci.py'. The breadcrumb navigation indicates the path '2-best-practices > 02-provide-top-level-comment > fibonacci.py'. The code content shows line 1 with a comment: `1 # This script calculates the Fibonacci sequence up to a given number`, and line 2 is empty.

Best Practices

2

Provide Examples in your prompt

By providing GitHub Copilot with examples of what you are expecting as an output or an input you are going to get a much more accurate result.



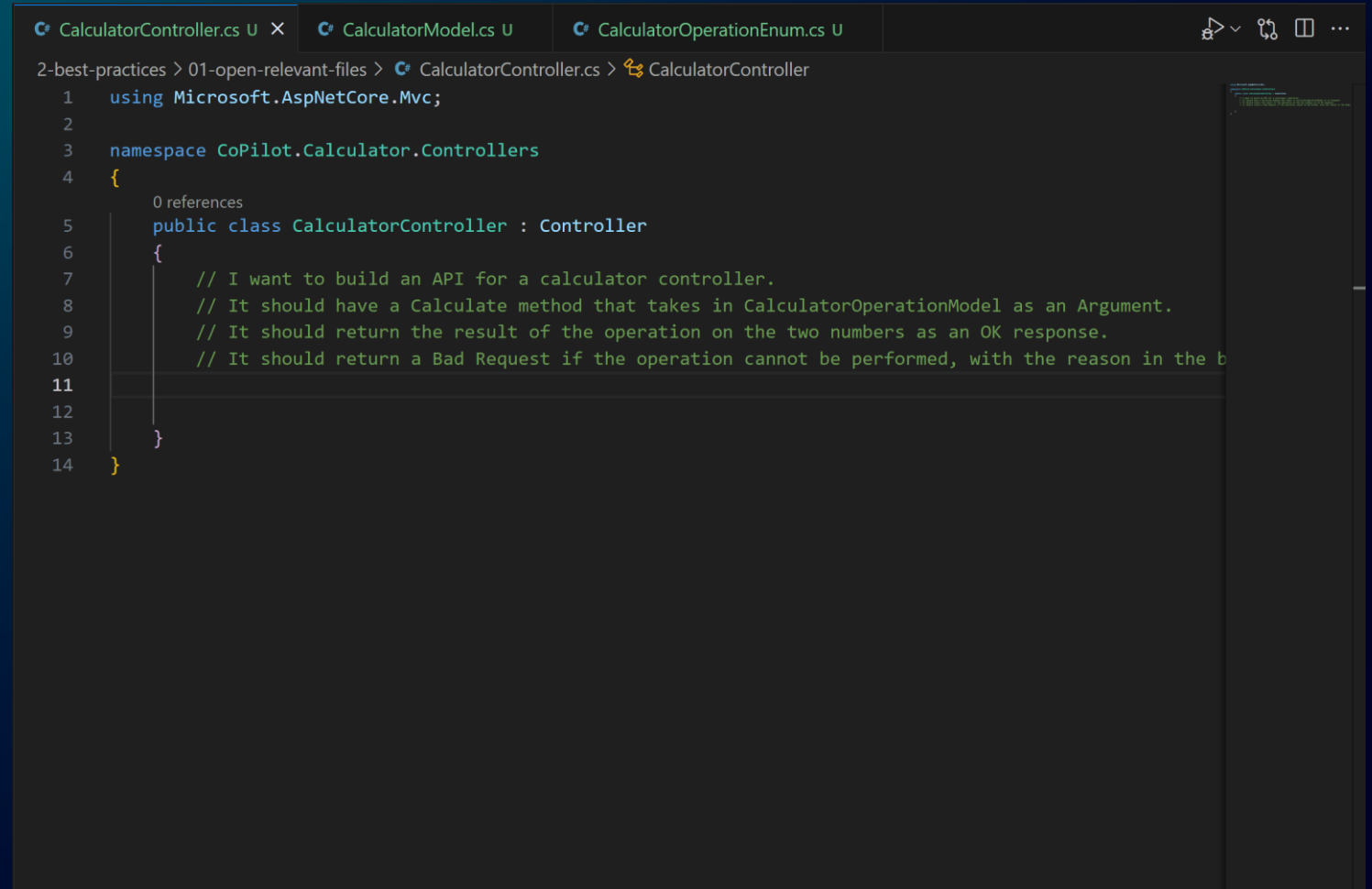
Best Practices

3

Keep Relevant Files Open

GitHub Copilot uses the context of your current workspace to provide relevant code suggestions.

This context includes the currently active file in your editor, as well as other files that are open in your workspace.



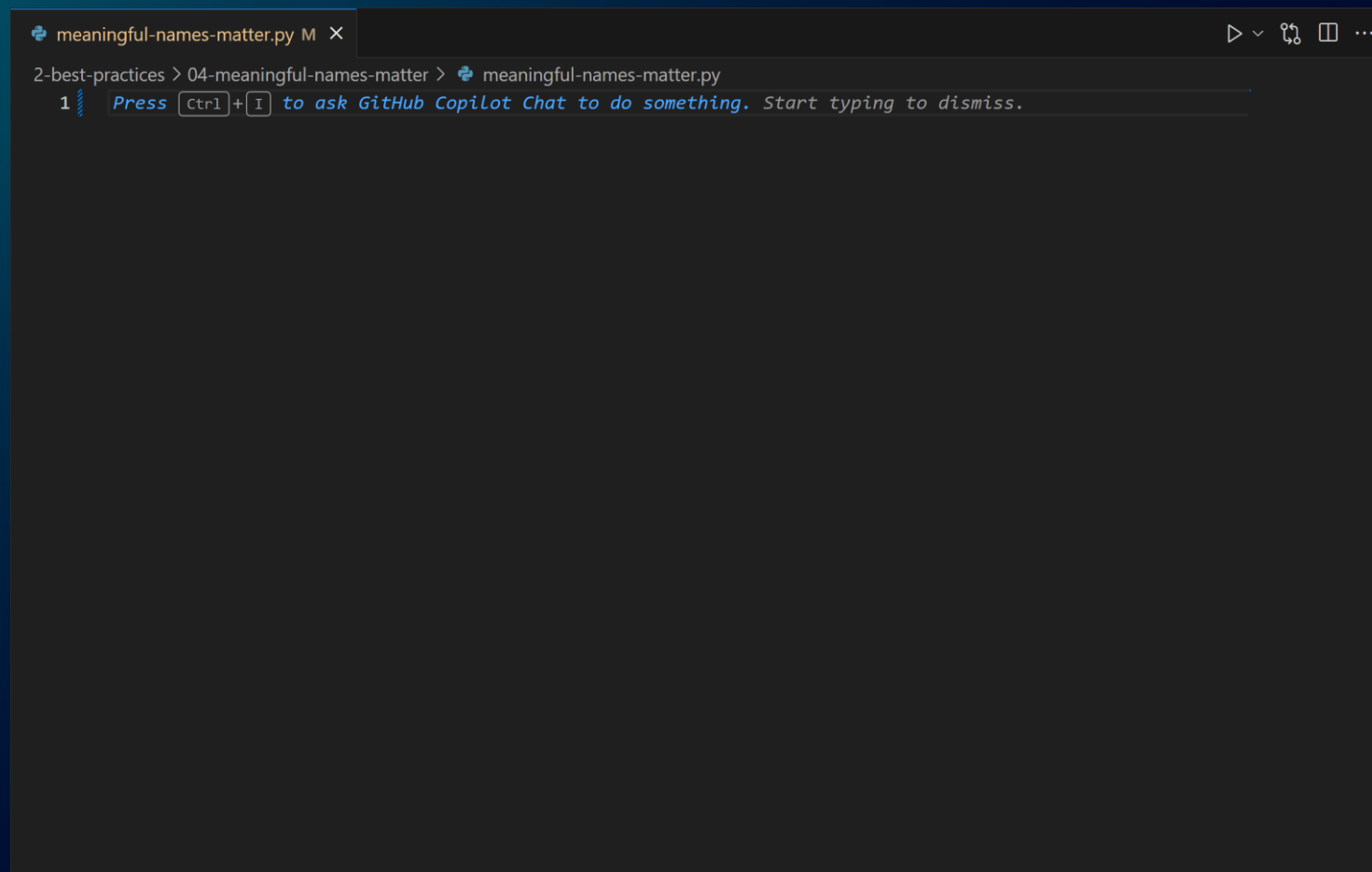
```
CalculatorController.cs U CalculatorModel.cs U CalculatorOperationEnum.cs U
2-best-practices > 01-open-relevant-files > CalculatorController.cs > CalculatorController
1 using Microsoft.AspNetCore.Mvc;
2
3 namespace CoPilot.Calculator.Controllers
4 {
5     0 references
6     public class CalculatorController : Controller
7     {
8         // I want to build an API for a calculator controller.
9         // It should have a Calculate method that takes in CalculatorOperationModel as an Argument.
10        // It should return the result of the operation on the two numbers as an OK response.
11        // It should return a Bad Request if the operation cannot be performed, with the reason in the b
12    }
13 }
14 }
```

Best Practices

4

Meaningful names matter

By using descriptive and concise method/variable names, you can enhance the accuracy and usefulness of Copilot's suggestions, leading to improved code quality and productivity.



Dev [Boost]

GitHub Copilot

Future DevBoost Training Topics

Additional trainings as part of the
Dev[Boost] program



Future training topics



Prompt Engineering

- Prompt Modifiers
- Role Prompting
- RAG like techniques



Foundations

- Slash Commands
- Variables
- Terminal Chat Participants
- Workspace Chat Participants
- VS Code Chat Participants
- Smart Actions



Best Practices

- General Guidelines to improve suggestions
- Using comments effectively
- Inline Chat & refactoring with Inline Chat
- Managing conversation threads
- Focusing GitHub Copilot on specific code
- Using #file variable



Use Cases

- Ideating
- Creating new workspaces
- Code reviews
- Language/Framework Conversion
- Generating IaC
- Working with internal libraries
- Working with SQL
- And more...



Dev [Boost]

GitHub Copilot

Thank you

Questions & Answers

